

Thoughts on AI-augmented Engineering and What's Next

This paper provides an overview of the current 'Five Times' acceleration initiative, and takes the lessons so far, combined with recent industry trends, to advocate for transformational SDLC / ways of working experimentation and changes at NAB.

An Update on Five Times

Five Times is the ambition within CIO Business, Private & Digital to accelerate delivery using AI and transforming ways of working to *do in a day what used to take a week*. Five Times applies to end-to-end delivery from requirements and engineering, through to release and governance. This follows on from successful proof of concept workshops held across BP&D teams in Q1, including workshops in India and Vietnam.

In late January six full-scale subdomain pilots kicked off for AI-enhanced engineering – this covered 54 squads across Digital Identity, HICAPS, BOSS (Merchant Acquiring), Ledger and Switch modernisation, and High Net Worth. The goal is to prove (and learn along the way) that AI-augmented teams, combined with transformed ways of working, can consistently deliver velocity improvements with sustainable quality outcomes.

Six subdomains is meaningful to keep the pilot controlled, but enough to get real data on what's working and what's not. There is no playbook for transforming delivery using AI, and it is a hyper-fast moving space, so experimenting and learning in a safe and iterative way is critical. Participants cover the whole SDLC, from product through to architecture, solution design, security and engineering. All participants have been trained, and the transformation team has produced extensive training material as well as ongoing mentorship. The pilot will run until May.

Notwithstanding that formal pilots have just begun, there are emerging signals from early-adopter teams who have incorporated a level of AI-augmentation:

- Feature elaboration (user stories, requirements etc) has greatly benefited from AI tooling. In some cases the elapsed time to elaborate has reduced by over 90%
- Code volume and related flow/impact scores have increased, ranging from 40%-110% when compared to pre-AI levels
- A strong negative correlation is emerging between the rate of feature delivery into production and the number of external dependencies (eg: handoffs / co-contributions etc) teams have. Where teams can solely deliver within themselves that are moving faster and more predictably, whereas the greater the number of external dependencies the lower the impact of AI-augmentation on throughput

The last point requires a deeper discussion, as solving this is really the key to unlocking the full potential that AI today, and what is to come in the very near future, can deliver.

The Need for Speed

Delivery of a feature, from idea to production, requires many steps and with NAB's broad array of systems it quite often involves more than one team – sometimes many. All steps must come together and synchronise for the feature to go live. Those steps include the usual requirements, engineering, and testing, but also the cloud configuration, the API governance, the security reviews, the prioritisation amongst domains and the end user change management. When multiple teams are involved the number of steps to navigate multiplies.

In a highly synchronous system, speeding up one part (eg: engineering with Cursor) without a commensurate increase in other parts does not yield any benefits, and only creates greater problems as backlogs and buffers fill. In pre-AI days wait times between hand offs were a hidden productivity tax, obscured from view by the relative length of other processes. But when the delivery of a feature can now be done in hours vs weeks those handoffs start to feel like an eternity and are immensely painful. Coordinating a large-scale system test, queuing for performance testing capacity, waiting a week for change approval are examples of 'standard' points of friction today, but whose effects only amplify in an AI world.

The implication is that if NAB is to gain any benefit from the potential and promise of AI, its end-to-end software delivery lifecycle and ways of working needs to be fundamentally rebuilt and calibrated around high-velocity throughput. To emphasise and add further weight to this call, AI tool vendors (Cursor, Anthropic etc) are releasing capability for multiple agents to plan, coordinate and execute long-running, high-volume code delivery – where thousands of lines of code will be written, committed, tested and repaired without any significant human intervention; a far more advanced position than the reality of individual developer augmentation we have with AI today. In this reality waiting a week to get your API spec approved would simply be a farce.

Reducing handoffs between teams and removing/automating offline batch-based governance is necessary to allow the entire end-to-end delivery machine to spin at significantly higher revolutions. The Agile constructs of scrums, the 'two pizza' squad and fortnightly sprints also need to be challenged. Pre-AI an entire team would own a system / product and jointly plan and coordinate delivery of its backlog. Now a single augmented engineer can build a whole system in a couple of weeks, if not faster. In this environment managing and coordinating our delivery resources into teams that are many times bigger than what is required will become incredibly wasteful. Deploying code every fortnight will also become highly risky, as the volume of change will be orders of magnitude greater than what it is today. Continuous delivery of small, incremental change is likely to be the safest approach.

Through the Five Times initiative we have been tackling the offline governance productivity impacts, through embedding the required controls into the engineering standards so that they are built-in during delivery and not an afterthought via a committee. This applies to security requirements and API governance – two important control points that can have a material effect on throughput. Open-sourcing NAB's internal source code is also another step being taken to reduce handoffs. Allowing external contribution to a codebase will decouple dependencies, especially where there is a velocity difference between teams. Open federation is in its early days and is as much a cultural challenge as it is a logistical one, but is an important step to improve overall speed of delivery.

The Future of Software Engineering

The speed at which AI tool vendors and AI model developers are maturing their products is breathtaking. The vendors use their own tools to improve their own tools, and are delivering value at a geometric rate. For example, the latest release of Claude Code was largely (circa 90%) written autonomously by Claude Code itself. The era of the positive feedback loop is upon us.

As mentioned earlier, highly-scaled agents is the looming next generation of the tools. Cursor ran an internal experiment which scaled out to 2,000 asynchronous yet coordinated agents in a single long-running session to write 1m lines of code and produce a (partially working, partially incomplete) web browser. Anthropic ran a lab-based experiment where a single, yet highly detailed, prompt drove a series of agents to coordinate and build a C compiler that was good enough to successfully compile the linux kernel (considered a premium benchmark stress test for compiler design). Whilst these were both clean room, vendor lab experiments, and neither was perfect, they do show that the newer models (OpenAI's 5.3 Codex, or Anthropic's 4.6 Opus) can enable large scale coordination across a fleet of agents to deliver complex outcomes. Both tool vendors are also enabling 'background' agents – where an agent can be kicked off to run in its own cloud environment (so not dependent on the engineer's laptop) and report back when done. Cursor will soon be enabling this feature so engineers can kick off background agents via a mobile app – so imagine engineers on the train sending prompts to agents for work to be completed by the time they get to the office, or code analysed and a plan drawn up, or further tests run. The possibilities are limitless.

The implications for the software industry, and NAB, are profound. The segregation of front end-only engineers, vs a back-end engineer, or a full stack, or a Java engineer vs a C# engineer is no longer relevant. As mentioned above the concept of scrums, squads and sprints also feels incongruent with where the tooling is heading. Importantly, the demarcation line between a product owner and an engineer will, in time, blur. Product owners will take on more engineering and engineers will become more product aware –

and we'll hire people for those roles on that basis. Ultimately they will merge into a single role.

The future engineer will be the one who is highly contextually aware, both of the customer and product needs, but also the technical environment. They'll craft the right prompt (noting that the 'right' prompt can sometimes be very lengthy, rich in detail, with multiple artefacts) and who ensures that the right quality controls are in place. AI still makes mistakes, and while it's getting better it's certainly not perfect. The engineer will then coordinate a set of agents to complete tasks, engaging in their planning and steering them in the right direction.

Today, we see engineers being augmented with AI. Where we are very quickly heading is that AI will be augmented with engineers. And that is the reality that we'll need to pivot towards.

So where to from here?

NAB is a very broad church of technology, from 60 year old IBM assembler, to the world's oldest Siebel instance, 500 SaaS solutions, hundreds of 3rd party packaged software, and tens of millions of lines of home-grown code from various generations.

If we believe that customer experience and customer advocacy is what will differentiate NAB and deliver sustainable competitive advantage, then we should have the goal of embracing AI at scale, and overhauling our ways of working to ensure that ***every engineer ships something meaningful to production every day***. Setting that level of ambition for delivery will require a strong focus on identifying customer needs / pain points, and will force us to deal head on with the blockers and points of friction within the SDLC.

As an immediate step the current six subdomain pilots will generate meaningful lessons. Everything right now is a theory, and these pilots will help us source real data points on what works and what doesn't. Those pilots are just as much focused on removing blockers and synchronisation problems as they are enabling engineers with AI, so meaningful change is expected as a result.

In parallel with the pilots we should establish a transformation execution group to sponsor further ways of working experiments and scaling / rollout of proven techniques. There is a rich array of shifts that need to be proved (or otherwise), such as smaller squads, continuous flow, product management, scaled agent-based delivery, quality control, change management etc. Through this, we should expect that the process of delivering software tomorrow will be vastly different from the process of delivering software today. No one knows the way there, but we need to start constructing the map as soon as possible, or we'll be left behind by those that do.